

UNIVERSIDAD CATÓLICA BOLIVIANA

Machine Learning · Grupo 3

OmniRec-Movies

Sistema Inteligente de Recomendación
de Películas — *OmniCine*

Informe Técnico Integral

Catálogo de modelos, arquitectura general, MLOps y plataforma web. Documenta el qué, el porqué, el para qué y el dónde de cada componente, sobre el dataset **MovieLens 25M** y la metodología **CRISP-DM**.

Dataset: MovieLens 25M

Stack: PyTorch · scikit-surprise ·
FAISS · FastAPI · Next.js 16

MLOps: pipeline + MLflow + Re-
gistry + Monitoreo

Fecha: 24 de junio de 2026

Resumen ejecutivo

OmniRec-Movies es un sistema de recomendación de películas construido siguiendo el ciclo **CRISP-DM** de extremo a extremo: desde el entendimiento del negocio y el análisis exploratorio, pasando por el modelado clásico y profundo, hasta el despliegue como una plataforma web (*OmniCine*) con una capa de **MLOps** reproducible. El proyecto combina **tres familias de modelos** que se reparten el trabajo según el contexto: factorización matricial clásica (**SVD**) para la recomendación personalizada, redes neuronales de embeddings (**MF+Bias / NeuMF**) para la similitud entre películas, y recuperación semántica con *transformers* (**MiniLM + FAISS + LLM**) para la búsqueda en lenguaje natural. Todos los modelos servidos están **versionados** con manifiestos SHA256, **trackeados** en MLflow y **monitoreados** frente a *drift* y degradación. Este informe documenta cada modelo en detalle, la arquitectura global, el funcionamiento de la capa MLOps y cómo cada pieza se traduce en una funcionalidad concreta de la web-app.

Cómo leer este documento

La Sección 2 ofrece el **panorama general**. La Sección 4 es el **catálogo detallado de modelos** (una ficha por modelo). La Sección 6 resume **qué se usa en producción y dónde**. La Sección 7 explica el **MLOps** en profundidad. La Sección 8 describe la **web-app**.

Índice

Resumen ejecutivo	i
1. Introducción y panorama general	1
1.1. Objetivo del sistema	1
1.2. Metodología: CRISP-DM por fases	1
1.3. Stack tecnológico	1
2. Arquitectura general del sistema	1
2.1. Componentes principales	1
3. Datos y preparación	2
4. Catálogo de modelos	2
4.1. Baseline de popularidad bayesiana	3
4.2. KNN-Baseline (filtrado colaborativo por vecindario)	3
4.3. SVD — Factorización matricial (modelo campeón clásico)	3
4.4. NMF — Factorización no negativa	4
4.5. AutoML — Selección de algoritmo e hiperparámetros	4
4.6. KMeans — Clústeres de comunidad (“personas”)	4
4.7. MF+Bias — Factorización matricial neuronal (Deep Learning de servicio)	4
4.8. NeuMF — Neural Collaborative Filtering	5
4.9. Encoder semántico — Sentence-Transformers (MiniLM)	6
4.10. FAISS — Índice vectorial	6
4.11. Capa generativa RAG — Claude (LLM)	6
5. Comparativas y resultados reales	6
5.1. Fase 3 — Modelos clásicos (<i>test</i> temporal warm-start)	6
5.2. Fase 4 — Deep Learning vs clásico (mismo <i>split</i> 70/15/15)	7
6. Qué se usa en producción y dónde	7

7. MLOps: cómo funciona	7
7.1. 1. Pipeline reproducible y orquestado	7
7.2. 2. Configuración como código	8
7.3. 3. Tracking de experimentos (MLflow)	8
7.4. 4. Registro y versionado de artefactos (SHA256)	8
7.5. 5. Evaluación con protocolo definido	8
7.6. 6. Monitoreo, <i>drift</i> y política de reentrenamiento	8
7.7. Despliegue reproducible (Docker)	9
8. La plataforma web (OmniCine)	9
9. Reproducibilidad y ejecución	9
10. Conclusiones	10

1 Introducción y panorama general

1.1 Objetivo del sistema

El objetivo es recomendar películas relevantes a cada usuario y permitir descubrir contenido mediante búsqueda en lenguaje natural, sobre el catálogo de **MovieLens 25M** (25 millones de calificaciones, ~62 000 películas, 1 128 *tags* del *Tag Genome*). El sistema atiende tanto a **usuarios registrados** (con historial) como a **invitados** (sin historial), por lo que la estrategia de modelado debe resolver explícitamente el problema de *cold-start*.

1.2 Metodología: CRISP-DM por fases

El proyecto se organiza en *notebooks* que corresponden a las fases de CRISP-DM, y en un código de producción (`src/`, `app/`) que materializa el despliegue y la operación.

Fase CRISP-DM	Notebook / Módulo	Aporte
1. Business + Data Understanding	01_Business_Understanding_and_EDA	Análisis exploratorio, comprensión del dominio.
2. Data Preparation	02_Data_Sampling_and_Cleaning	Muestreo estratificado, limpieza, <i>parquets</i> .
3. Modeling (clásico)	03_ML_Baseline_AutoML	Baseline, KNN, SVD, NMF, AutoML, KMeans.
4. Modeling (profundo)	04_DeepLearning_Embeddings	MF+Bias y NeuMF (PyTorch).
4. RAG / recuperación	05_Semantic_Search_RAG	Encoder MiniLM, FAISS, capa generativa.
4/5/6. MLOps + Deployment	<code>src/</code> , <code>app/</code> , <code>docker</code>	Pipeline, MLflow, registry, monitoreo, API+web.

1.3 Stack tecnológico

- **Modelado:** scikit-surprise (SVD, KNN, NMF), scikit-learn (KMeans), **PyTorch 2.4** (MF+Bias, NeuMF), Sentence-Transformers + **FAISS** (recuperación), Anthropic Claude (capa generativa).
- **MLOps:** pipeline propio en `src/`, **MLflow** (tracking), *model registry* con manifiestos SHA256, monitoreo de *drift*.
- **Servicio:** **FastAPI** (backend, puerto 8000), **Next.js 16** (frontend *OmniCine*, puerto 3000), **Docker Compose** (orquestración).

2 Arquitectura general del sistema

El sistema separa con claridad el **entrenamiento** (offline, reproducible) del **servicio** (online, de baja latencia). Los modelos se entrenan en notebooks o en el pipeline, se persisten como **artefactos versionados**, y el backend los carga para servir inferencia a la web-app.

2.1 Componentes principales

- **Datos** (`data/`): *parquets* intermedios generados por los notebooks 02/03. El dataset canónico de servicio es `ratings_knn_10pct.parquet` (2,47 M de calificaciones).
- **Entrenamiento** (`notebooks/`, `src/`): los notebooks documentan el modelado; el pipeline `src/pipeline.py` lo reproduce de forma operativa.
- **Artefactos + Registry** (`models/`): pesos y factores serializados, versionados en `models/registry/` con manifiestos SHA256.
- **Backend** (`app/backend/`): **FastAPI**. `ml/engine.py` sirve recomendación y similitud; `ml/semantic.py` sirve búsqueda semántica y RAG.

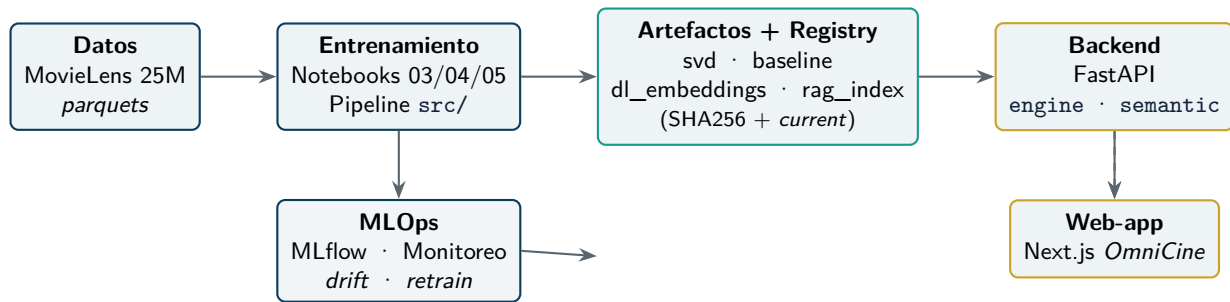


Figura 1: Flujo extremo a extremo: los datos alimentan el entrenamiento (offline), que produce artefactos versionados en el *registry*; el backend los sirve a la web-app. La capa MLOps acompaña todo el ciclo.

- **Frontend** (app/web-app/): Next.js 16 *OmniCine*, consume la API REST.
- **MLOps** (src/, config/, docker): pipeline, MLflow, registry, monitoreo y despliegue reproducible.

3 Datos y preparación

El notebook 02 realiza un **muestreo estratificado de usuarios** para obtener subconjuntos manejables sin sesgar la distribución de actividad, y persiste *parquets* con compresión *snappy*:

Artefacto	Tamaño	Uso
ratings_knn_10pct.parquet	2,47 M filas	Dataset canónico de servicio (entrenamiento DL y SVD operativo).
movies_prepared_60pct.parquet	55 113 filas	Catálogo (movieId, title, genres).
genome_scores_prepared_60pct.parquet	15,58 M filas	Relevancia tag-película (descriptores RAG).
genome_tags.parquet	1 128 filas	Diccionario de tags del <i>Tag Genome</i> .
item_clusters.parquet / user_clusters.parquet	—	Clústeres y estadísticos por película/usuario.

Contrato de datos y Docker

La imagen Docker incluye solo los *parquets* necesarios para servir (vía *.dockerignore*); por eso el dataset operativo es *ratings_knn_10pct.parquet* y no los *samples* de exploración. El pipeline resuelve automáticamente el primer *parquet* de *ratings* disponible (*src/data.py:resolve_ratings_path*).

4 Catálogo de modelos

Esta sección documenta **cada modelo** con una ficha uniforme: qué es, cómo funciona, por qué se eligió, para qué sirve y dónde se usa (en documentación/notebook y en la web-app). El estado se etiqueta como **EN PRODUCCIÓN**, **SOLO DOCUMENTACIÓN** o **PROCESO / SELECCIÓN**.

4.1 Baseline de popularidad bayesiana

Baseline — Bayesian Weighted Rating

EN PRODUCCIÓN

Qué es. Ranking no personalizado que pondera la media de una película con un *prior* global, evitando que películas con pocos votos dominen.

Cómo funciona. $\text{score} = \frac{n}{n+m} \bar{r} + \frac{m}{n+m} C$, donde $\bar{r}$ es la media de la película, n su número de votos, C la media global y m un umbral (cuantil de votos).

Por qué. Es robusto, interpretable y resuelve el *cold-start* de usuario: cuando no hay historial, no hay nada que personalizar.

Para qué / Dónde. *Documentación:* notebook 03. *Web-app:* recomendaciones para invitados y *pool* de candidatos a re-ranear. *Backend:* `engine.popular()`, artefacto `baseline_scores.pkl`.

Métricas (test temporal, nb03). RMSE 1,009 · MAE 0,792 (línea base a superar).

4.2 KNN-Baseline (filtrado colaborativo por vecindario)

KNN-Baseline (item-based, Pearson + ALS)

SOLO DOCUMENTACIÓN

Qué es. Filtrado colaborativo basado en vecinos: predice el rating combinando ítems similares al objetivo, corrigiendo por sesgos con *baselines* ALS.

Cómo funciona. Similitud `pearson_baseline` entre películas (`user_based=false`), $k = 40$ vecinos, *min support* 5.

Por qué (y por qué NO se sirve). Es un competidor clásico fuerte, pero quedó por detrás del SVD en RMSE y es costoso en memoria/latencia; no se persistió para producción.

Para qué / Dónde. *Documentación:* notebook 03 (comparación). *Web-app:* no se usa.

Métricas (test temporal, nb03). RMSE 0,893 · MAE 0,669.

4.3 SVD — Factorización matricial (modelo campeón clásico)

SVD (Matrix Factorization)

EN PRODUCCIÓN

Qué es. Factorización de la matriz usuario-ítem en factores latentes; predice $\hat{r}_{ui} = \mu + b_u + b_i + q_i^\top p_u$.

Cómo funciona. 50 factores, 20 épocas, `lr=0.005`, `reg=0.02` (scikit-surprise). Para **usuarios nuevos** aplica *folding-in*: estima p_u resolviendo un *ridge* sobre los factores de los ítems que el usuario calificó (`engine._fold_in`).

Por qué. Mejor RMSE entre los clásicos, ligero, rápido y con *folding-in* barato $\Rightarrow$ resuelve el *cold-start* de usuario en vivo. Es el **modelo de inferencia operativo** del recomendador.

Para qué / Dónde. *Documentación:* notebook 03, etapa `train` del pipeline. *Web-app:* página **Para Ti** (recomendaciones), predicción de afinidad en la ficha. *Backend:* `engine.recommend()`, `predict_rating()`; artefacto `svd_model.pkl`.

Métricas. Test temporal nb03: RMSE 0,865 · MAE 0,656 (campeón). Mismo *split* que el DL (nb04): RMSE 0,819 · MAE 0,621.

4.4 NMF — Factorización no negativa

NMF (Non-negative Matrix Factorization)

SOLO DOCUMENTACIÓN

Qué es / Cómo funciona. Factorización con restricción de no negatividad sobre los factores, evaluada como alternativa al SVD.

Por qué (NO se sirve). Peor RMSE que el SVD; solo se mantiene como punto de comparación.

Para qué / Dónde. *Documentación:* notebook 03. *Web-app:* no se usa.

Métricas (test temporal, nb03). RMSE 0,936 · MAE 0,713.

4.5 AutoML — Selección de algoritmo e hiperparámetros

AutoML (auto-surprise TPE / GridSearchCV)

PROCESO / SELECCIÓN

Qué es. Búsqueda automática sobre SVD/NMF/BaselineOnly (optimización bayesiana TPE, con *fallback* a GridSearchCV).

Por qué. Justifica de forma sistemática la elección del SVD y sus hiperparámetros, en vez de fijarlos a mano.

Para qué / Dónde. *Documentación:* notebook 03. Es un **proceso de selección**, no un modelo servible; su salida (elegir SVD) sí se usa.

4.6 KMeans — Clústeres de comunidad (“personas”)

KMeans ($k = 6$) sobre factores latentes

EN PRODUCCIÓN

Qué es. Agrupamiento de usuarios (y de películas) en el espacio de factores latentes del SVD, para construir “comunidades” o *personas*.

Cómo funciona. $k = 6$ clústeres sobre p_u (usuarios) y q_i (películas); cada clúster recibe un perfil de géneros por *lift*. Para un usuario nuevo se asigna por **centroide euclidiano más cercano** a su vector *folded-in*.

Por qué / Para qué. Aporta una *feature* de identidad/comunidad y explica las recomendaciones.

Dónde. *Documentación:* notebook 03. *Web-app:* página **Perfil** (“tu comunidad”). *Backend:* `engine.persona_for()`; centroides y perfiles embebidos en `svd_model.pkl`.

4.7 MF+Bias — Factorización matricial neuronal (Deep Learning de servicio)

MF+Bias (PyTorch)

EN PRODUCCIÓN

Qué es. Análogo neuronal del SVD: *embeddings* de usuario e ítem cuyo producto punto, más sesgos, predice el rating, con salida acotada al rango $[0,5; 5]$ por una sigmoide escalada.

Cómo funciona. $\hat{r}_{ui} = \sigma(q_i^\top p_u + b_u + b_i + \mu)$ escalado; *embeddings* de dimensión 32, optimizador Adam, *early stopping* sobre Val RMSE (guarda la mejor época).

Por qué. En una comparación **justa** (mismo *split*, sesgos, salida acotada) **supera al SVD**. Sus *embeddings* de ítem capturan estructura semántica sin supervisión y **no tienen cold-start** (las películas son fijas).

Para qué / Dónde. *Documentación:* notebook 04, etapa `train_dl` del pipeline. *Web-app:*

sección “**Si te gustó esta, mira...**” en la **ficha de película** (*películas similares*). *Backend*: `engine.similar_dl()`; artefacto `dl_item_embeddings.pkl` (servido), `mf_model.pt` (pesos).

Métricas (test, mismo *split* nb04). RMSE **0,8046** · MAE 0,6113 (mejor de todos). Reproducible en notebook, local y Docker.

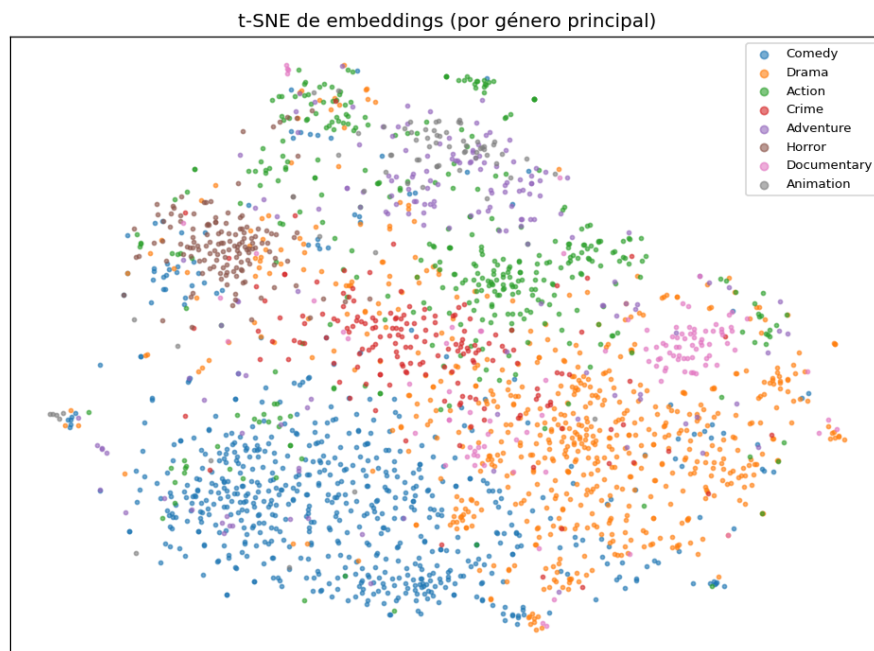


Figura 2: Proyección t-SNE de los *embeddings* de película aprendidos, coloreados por género principal. La estructura por géneros emerge **sin** que el modelo haya visto los géneros: evidencia de que el espacio latente es semánticamente significativo.

4.8 NeuMF — Neural Collaborative Filtering

NeuMF (GMF $\oplus$ MLP)

SOLO DOCUMENTACIÓN

Qué es. La arquitectura *Neural Collaborative Filtering* canónica: una rama **GMF** (producto elemento a elemento) y una rama **MLP** (no lineal) fusionadas, con sesgos.

Cómo funciona. Dos juegos de *embeddings* (GMF y MLP), MLP con capas 64-32-16 + *dropout*, fusión y cabeza lineal; misma salida acotada y *early stopping*.

Por qué (NO se sirve). Iguala al SVD pero **no supera** al MF+Bias en este volumen, y su brazo MLP hace que el vector de usuario sea no lineal $\Rightarrow$ **no admite *folding-in* barato** para usuarios nuevos. Queda entrenado y versionado como comparación.

Para qué / Dónde. *Documentación*: notebook 04, etapa `train_dl`. *Web-app*: no se sirve.

Métricas (test, nb04). RMSE 0,8196 · MAE 0,6208 (~2,8M parámetros, vs 1,4M del MF+Bias).

4.9 Encoder semántico — Sentence-Transformers (MiniLM)

Encoder paraphrase-multilingual-MiniLM-L12-v2

EN PRODUCCIÓN

Qué es. Modelo *transformer* preentrenado (inferencia, CPU) que convierte texto en *embeddings* densos. Multilingüe: permite consultas en español sobre descriptores en inglés.

Cómo funciona. Codifica un **descriptor** por película (título + géneros + *top 12 tags* genome con relevancia $\geq 0,5$) en un vector L2-normalizado.

Por qué. Habilita búsqueda por **significado** (no por coincidencia de texto), en lenguaje natural y multilingüe, sin entrenar nada.

Para qué / Dónde. *Documentación:* notebook 05, etapa `index` del pipeline. *Web-app:* página **Buscar**. *Backend:* `ml/semantic.py`; artefacto `models/rag_index/`.

4.10 FAISS — Índice vectorial

FAISS (inner product = coseno)

EN PRODUCCIÓN

Qué es / Cómo funciona. Índice de búsqueda de vecinos sobre los vectores normalizados del encoder; producto interno equivalente a similitud coseno. Un *re-ranking* mezcla la similitud semántica con un *prior* de popularidad ($\alpha = 0,25$).

Por qué / Para qué. Recuperación rápida de candidatos relevantes a la consulta; el *re-ranking* equilibra relevancia y calidad.

Dónde. *Documentación:* notebook 05, etapa `index`. *Web-app:* página **Buscar**. *Backend:* `ml/semantic.py`.

4.11 Capa generativa RAG — Claude (LLM)

RAG generativo (claude-haiku-4-5)

EN PRODUCCIÓN

Qué es. *Retrieve-Augment-Generate*: recupera películas relevantes con FAISS y redacta una recomendación en lenguaje natural con un LLM.

Cómo funciona. El contexto recuperado se inyecta en un *prompt*; la generación es opcional (`generate=true`) y degrada a una plantilla si no hay API key.

Por qué / Para qué. Convierte la búsqueda en una experiencia **conversacional**.

Dónde. *Documentación:* notebook 05. *Web-app:* **Buscar** con respuesta generada. *Backend:* `src/generate.py`, expuesto en `/api/search/semantic?generate=true`.

5 Comparativas y resultados reales

5.1 Fase 3 — Modelos clásicos (*test* temporal warm-start)

Modelo	RMSE ↓	MAE ↓	Estado
SVD (campeón)	0,865	0,656	EN PRODUCCIÓN
KNN-Baseline (item)	0,893	0,669	SOLO DOCUMENTACIÓN
AutoML → BaselineOnly	0,923	0,704	PROCESO / SELECCIÓN
NMF	0,936	0,713	SOLO DOCUMENTACIÓN
Baseline (Pop. bayesiana)	1,009	0,792	EN PRODUCCIÓN

5.2 Fase 4 — Deep Learning vs clásico (mismo *split* 70/15/15)

Modelo	Familia	Test RMSE ↓	Test MAE ↓
MF+Bias (DL)	Deep Learning	0,8046	0,6113
SVD (clásico, mismo <i>split</i>)	Factorización	0,8194	0,6212
NeuMF (DL)	Deep Learning	0,8196	0,6208

Rigor metodológico

Las dos tablas usan **protocolos de evaluación distintos** (la Fase 3 usa un *split* temporal warm-start; la Fase 4 un *split* 70/15/15 sobre `ratings_knn_10pct`), por lo que sus RMSE **no son comparables entre fases**. La conclusión clave de la Fase 4 es que, en una comparación *justa* (mismo *split*, sesgos, salida acotada), **el Deep Learning iguala o supera al clásico**; la diferencia que se atribuía antes a “superioridad del SVD” era un artefacto de comparar datasets distintos.

6 Qué se usa en producción y dónde

El sistema combina los modelos según el contexto, ubicando cada uno donde su fortaleza no choca con el *cold-start*.

Modelo	Artefacto	Endpoint	Página web / Por qué
Pop. bayesiana	<code>baseline_scores.pkl</code>	<code>/recommendations/*</code>	Invitados / <i>cold-start</i> .
SVD + <i>folding-in</i>	<code>svd_model.pkl</code>	<code>/recommendations/home</code> , <code>/affinity</code>	Para Ti , afinidad: personalización ligera.
KMeans personas	(en <code>svd_model.pkl</code>)	<code>/meta/personas</code>	Perfil : comunidad del usuario.
MF+Bias (DL)	<code>dl_item_embeddings.pkl</code>	<code>/movies/{id}/similar?</code>	Ficha de película : “peliculas similares” (sin <i>cold-start</i>).
Encoder + FAISS	<code>rag_index/</code>	<code>/search/semantic</code>	Buscar : lenguaje natural.
RAG (Claude)	— (API)	<code>/search/semantic?generate=true</code>	Buscar : respuesta redactada.

Diseño defensivo

La similitud DL tiene **fallback transparente**: si `dl_item_embeddings.pkl` no existe o una película no fue modelada, `engine.similar_dl()` cae al espacio latente del SVD sin romper nada. La capa generativa degrada a plantilla sin API key. El sistema nunca falla por un artefacto ausente.

7 MLOps: cómo funciona

La capa MLOps (CRISP-DM Fase 4) hace que el ciclo de vida de los modelos sea **reproducible, versionado, trazable y monitoreado**. Se apoya en seis pilares.

7.1 1. Pipeline reproducible y orquestado

`src/pipeline.py` encadena **etapas idempotentes** dirigidas por configuración (semilla fija = 42):

prepare → train → train_dl → baseline → index → evaluate → register → monitor

```
python -m src.pipeline all           # ciclo completo
python -m src.pipeline train_dl      # solo Deep Learning
make pipeline                        # equivalente dentro de Docker
```

7.2 2. Configuración como código

Nada de números mágicos: todo vive en `config/*.yaml`.

- `pipeline.yaml` — rutas, hiperparámetros de SVD y `deep_learning`, protocolo de evaluación, MLflow.
- `rag.yaml` — encoder, índice FAISS, *re-ranking*, capa generativa.
- `monitoring.yaml` — umbrales de alerta y política de reentrenamiento.

7.3 3. Tracking de experimentos (MLflow)

Cada ejecución de `train_dl` y `evaluate` abre un *run* de MLflow que registra parámetros, métricas y *tags*. **Verificado** en Docker: el experimento `omnirec-fase4-mlops-rag` contiene los *runs* `train_dl` (con `mf_bias_test_rmse=0.8046`, `neumf_test_rmse=0.8191`, parámetros y tiempos) y `evaluate`. Degrada a *no-op* si MLflow no está, sin romper el pipeline.

7.4 4. Registro y versionado de artefactos (SHA256)

`src/registry.py` versiona cada artefacto en `models/registry/<nombre>/<versión>/` con un **manifiesto SHA256** (integridad y trazabilidad), `dataset_version`, métricas y linaje (`parent`). El alias `current.json` apunta a la versión en producción, habilitando **promoción y rollback**. **Verificado** (pipeline completo en Docker): cuatro artefactos versionados.

Artefacto	Versión <i>current</i>	Métricas en el manifiesto
<code>svd_model</code>	2026.06.24-1	recomendador (RMSE, NDCG10).
<code>baseline_scores</code>	2026.06.24-1	estrategia bayesiana.
<code>rag_index</code>	2026.06.24-1	recuperación (Pk, cobertura).
<code>dl_embeddings</code>	2026.06.24-1	Test RMSE 0,8046, MAE 0,6113, linaje.

7.5 5. Evaluación con protocolo definido

`src/evaluate.py` mide el recomendador (`rmse`, `mae`, `hitrate@10`, `ndcg@10`, vía *folding-in*) y la recuperación (`mean_precision@k`, `coverage_genome`). **Verificado** en Docker: `rmse=0.8823`, `ndcg@10=0.2488`, `P@k=0.8833`, cobertura 0,25.

7.6 6. Monitoreo, *drift* y política de reentrenamiento

`src/monitor.py` genera un reporte JSON que combina calidad, *drift* y degradación, y dispara alertas contra `monitoring.yaml`:

Señal monitoreada	Umbral
Caída de NDCG10 vs versión <i>current</i> (SVD)	> 5 %
Subida de Test RMSE del modelo DL vs <i>current</i>	> 5 %
<i>Drift</i> de la distribución de <i>ratings</i> (PSI antiguo vs reciente)	> 0,2
Cobertura del recomendador / genome	< 0,6
Frescura de datos (días desde el último <i>rating</i>)	> 30

La **política de reentrenamiento** (disparadores) considera 50 000 *ratings* nuevos acumulados, un *schedule* mensual de respaldo y un **guardrail**: solo promueve a *current* si las métricas no empeoran. **Verificado**: el reporte incluye el bloque `deep_learning` (con `rmse_rise_pct_vs_current`) y detectó correctamente 3 alertas reales del dataset estático (*drift* de *ratings*, cobertura genome y antigüedad de datos).

7.7 Despliegue reproducible (Docker)

El `docker-entrypoint.sh` construye artefactos de forma **idempotente** en el primer arranque (índice, SVD, baseline, y ahora el modelo DL vía `train_dl + export + register`). Las imágenes fijan dependencias con su justificación (p.ej. `numpy<2` por `scikit-surprise`, `torch==2.4` por estabilidad). Backend, MLflow y Jupyter comparten la misma imagen (paridad de entorno).

Cobertura MLOps por modelo

Los cuatro artefactos de servicio (SVD, baseline bayesiano, *embeddings* DL, índice RAG) están **en el pipeline, trackeados en MLflow y versionados** con manifiestos SHA256. El modelo Deep Learning quedó completamente integrado al ciclo MLOps, reproducible en los tres entornos (notebook, local y Docker) con el mismo Test RMSE **0,8046**.

8 La plataforma web (OmniCine)

El frontend Next.js consume la API REST de FastAPI. El recorrido del usuario activa distintos modelos:

Página	Función	Modelo(s)
/cartelera	Explorar el catálogo (filtros, orden)	Popularidad / catálogo
/buscar	Búsqueda en lenguaje natural	Encoder + FAISS + RAG
/para-ti	Recomendaciones personalizadas	SVD + <i>folding-in</i> / baseline
/pelicula/[id]	Ficha + “películas similares” + afinidad	MF+Bias (DL) + SVD
/perfil	“Tu comunidad”	KMeans personas
/login, /registro	Autenticación	—

Flujo invitado → **usuario**. Los invitados acumulan *feedback* en `localStorage`; al iniciar sesión se migra al backend (`/ratings/sync`), de modo que las recomendaciones personalizadas (SVD *folding-in*) y la *persona* (KMeans) se activan inmediatamente sobre el historial recién migrado.

9 Reproducibilidad y ejecución

```
# Stack completo (backend + web + MLflow)
make up-all                                # web: localhost:3000 / MLflow: localhost:5001

# Pipeline MLOps de extremo a extremo (dentro del backend)
docker compose run --rm backend python -m src.pipeline all

# Reentrenar y versionar solo el modelo Deep Learning
docker compose run --rm backend python -m src.pipeline train_dl
docker compose run --rm backend python -m src.pipeline register

# Notebooks (Jupyter sobre la misma imagen)
make jupyter                                # http://localhost:8888
```

Garantías de reproducibilidad

Semilla fija (42) en todo el pipeline, dependencias fijadas con su motivo, *splits* deterministas, hashes SHA256 por artefacto y configuración como código. El mismo Test RMSE (0,8046) se obtiene en notebook, en local y en Docker.

10 Conclusiones

- El sistema integra **tres familias de modelos** (factorización clásica, redes neuronales de *embeddings* y recuperación semántica con *transformers* + LLM), ubicando cada una donde su fortaleza es máxima y el *cold-start* no la penaliza.
- En una comparación **justa**, el Deep Learning (MF+Bias) **igual o supera** al clásico (RMSE 0,8046 vs 0,8194), y sus *embeddings* capturan estructura semántica real, hoy explotada en “películas similares”.
- **Todo lo que se sirve está en MLOps y versionado**: pipeline reproducible, *tracking* en MLflow, *registry* con manifiestos SHA256 y linaje, y monitoreo de *drift* y degradación, verificado de extremo a extremo en Docker.
- La web-app *OmniCine* traduce cada modelo en una funcionalidad concreta y comprensible para el usuario final, con *fallbacks* defensivos que garantizan disponibilidad.